

ephemnous

Escalonamento por aprendizado de máquina para data centers em órbita baixa

Karina Queiroz de Gennaro (RM570928)

Luis Felipe Bardi (RM569479)

Beatriz de Oliveira Ossola Ribeiro (RM570190)

FIAP Global Solution

QUERO CONCORRER

9 de junho de 2026

Resumo

Data centers em órbita baixa deixaram de ser ficção: em 2025, projetos como Starcloud e o Google Project Suncatcher colocaram (ou anunciaram) computação em órbita. Lá, a energia solar oscila (o satélite entra na sombra da Terra a cada ~ 95 min) e o calor só sai por radiação, porque no vácuo não há convecção. Apresentamos o **ephemnous**, um escalonador que prevê a energia e a folga térmica das próximas janelas e decide o que computar: rodar no sol, fazer *checkpoint* antes do eclipse e dar *throttle* para não superaquecer. Um *forecaster* de *gradient boosting* prevê energia e folga térmica; um controlador preditivo (MPC) decide a partir das previsões. Medimos com honestidade: a previsão supera um baseline reativo justo de forma **robusta apenas no regime energia-crítico** (ganho de 20–25% em *jobs* concluídos, intervalo de confiança de 95% acima de zero em três blocos de sementes) e **empata quando há folga de bateria**. O sistema roda ponta a ponta: ESP32 simulado no Wokwi, backend FastAPI, banco PostgreSQL e *dashboard*.

1 Introdução

A demanda por computação cresce mais rápido que a capacidade de alimentá-la e refrigerá-la na Terra. Uma resposta emergente é levar data centers para a órbita, onde há energia solar abundante e um sumidouro térmico frio. O conceito saiu do papel em 2025 com iniciativas como Starcloud e Google Project Suncatcher, mas não existe um demonstrador acessível do *escalonamento* dessa carga.

O ambiente orbital impõe dois problemas sem análogo no solo:

- **Energia intermitente.** Em órbita baixa o período é de cerca de 95 minutos e parte dele é passada na sombra da Terra (eclipse), quando o painel não gera nada.
- **Calor só por radiação.** No vácuo não há convecção; o calor gerado pela computação só sai irradiando, de forma lenta e não linear (lei de Stefan-Boltzmann).

Decidir o que computar a cada instante, sob essas restrições, é um problema de escalonamento ciente de energia e temperatura. O *ephemnous* ataca esse problema e, mais importante, *mede honestamente* quando a previsão agrega valor. O projeto se alinha aos ODS 9 (indústria, inovação e infraestrutura) e 13 (ação climática).

A contribuição não é “nossa IA vence sempre”. É um sistema completo, reproduzível e barato, com uma avaliação rigorosa que mostra *onde* prever ganha de reagir, e onde não ganha.

2 Desenvolvimento

2.1 Visão geral da solução

O ephemnous é um laço fechado: um nó de borda reporta telemetria; o *backend* avança a física, prevê o futuro próximo, decide a ação e devolve o comando. A física orbital, térmica e de bateria vive em um único módulo Python, compartilhado pela demonstração ao vivo, pela frota simulada e pela geração de dados de treino. Isso elimina divergência entre demo e treino.

2.2 Arquitetura

A Figura 1 mostra os componentes. O nó de borda é um ESP32 simulado no Wokwi: lê um potenciômetro (irradiância solar e gatilho de eclipse) e um botão, atua um LED por PWM (a carga de computação visível) e troca dados por HTTP. O backend FastAPI é o dono da escrita no PostgreSQL e roda o *forecaster* e o MPC. Uma frota simulada de N nós e a geração de dados usam a mesma física.

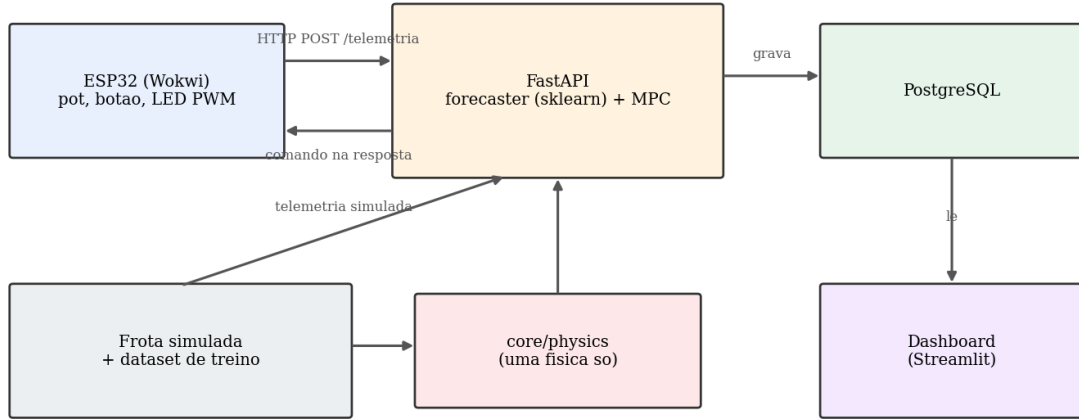


Figura 1: Arquitetura do ephemnous. A física fica só no backend; o ESP32 é um nó fino que sente e atua.

Decisão: onde a física mora. Optamos por mantê-la *só no backend* (não no firmware), o que evita duplicar e dessincronizar a física entre C++ e Python. É também fisicamente honesto: um satélite real não “calcula” a irradiância que recebe; isso é o mundo. O ESP32 contribui com leitura de sensor e atuação.

2.3 Simulador e física

O relógio orbital usa a terceira lei de Kepler, $T = 2\pi\sqrt{a^3/\mu}$, com $a = R_{\oplus} + h$. A fração de órbita em eclipse, para um ângulo β , é

$$f_{ecl} = \frac{1}{\pi} \arccos\left(\frac{\sqrt{h^2 + 2R_{\oplus}h}}{(R_{\oplus} + h) \cos \beta}\right).$$

A temperatura segue um balanço térmico de massa concentrada, integrado por Euler:

$$C \frac{dT}{dt} = Q_{comp} + Q_{solar} - \varepsilon \sigma A (T^4 - T_{sink}^4),$$

onde σ é a constante de Stefan-Boltzmann. A bateria integra a potência líquida, $dE/dt = P_{gen} - P_{draw}$, e o estado de carga é $SoC = E/E_{cap}$. O núcleo desse passo é direto:

```

1 q_solar = p.absorptivity * p.s_solar_w_m2 * p.a_panel_m2 * eff
2 q_comp  = load * p.q_comp_max_w
3 q_rad   = p.emissivity * SIGMA * p.a_rad_m2 * (state.temp_k**4 - p.t_sink_k**4)
4 temp    = state.temp_k + (p.dt_s / p.c_thermal_j_k) * (q_comp + q_solar - q_rad)
5
6 p_gen   = p.panel_eff * p.s_solar_w_m2 * p.a_panel_m2 * eff
7 p_draw  = p.p_base_w + load * p.p_comp_max_w
8 energy_j = clamp(state.soc_frac * e_cap_j + (p_gen - p_draw) * p.dt_s, 0.0, e_cap_j)
9 soc     = energy_j / e_cap_j

```

2.4 Coleta de dados e banco

O ESP32 envia telemetria por POST /telemetry e recebe o comando na resposta do mesmo POST (um *round-trip* fecha o laço; escolhemos HTTP em vez de MQTT porque um nó real mais a frota simulada não justifica um *broker*). O PostgreSQL guarda nós, telemetria, estado físico, previsões, decisões e *jobs*. As unidades são canônicas: potência em Watts, temperatura em Kelvin, frações em $[0, 1]$, com a conversão para Kelvin feita no firmware antes do envio.

2.5 Modelos de IA

Forecaster (o aprendizado). Um `HistGradientBoostingRegressor` (scikit-learn) prevê, para horizontes de 1 a 5 passos, a potência disponível e a folga térmica. As *features* usam apenas informação em t ou antes (defasagens de potência, temperatura e carga; estatísticas móveis; *features* cíclicas de fase). Contra vazamento (*leakage*): os rótulos vêm de um *rollout* limpo, as *features* vêm de observações com ruído de sensor, e a divisão treino/validação é por episódio. Um fator estocástico de apontamento/atitude (passeio aleatório) torna o futuro não determinável apenas pela efeméride.

Escalonador (MPC heurístico). O controlador consome as previsões e decide entre rodar, adiar, fazer *checkpoint* ou *throttle*. O baseline “reativo” é o *mesmo código* com horizonte zero, o que torna a comparação justa por construção.

```

1 if fut_margin < MARGIN_CRIT_K:
2     return Decision("throttle", 40, ...)      # folga termica critica prevista
3 if eclipse_soon and state.soc_frac < CHECKPOINT_SOC:
4     return Decision("checkpoint", 0, ...)     # salva antes de perder energia
5 if state.power_avail_w < p.p_base_w and state.soc_frac < SOC_LOW:
6     return Decision("defer", 0, ...)          # sem energia, adia
7 if fut_margin < MARGIN_LOW_K:
8     return Decision("throttle", 128, ...)     # folga moderada, meia carga
9 return Decision("run", 255, ...)              # tudo folgado, plena carga

```

2.6 Decisões do grupo

- **Arquitetura limpa, sem DDD.** Camadas *core* (puro), *services* (orquestração) e *infra* (FastAPI, Postgres, ML), com a regra de dependência apontando para dentro.
- **Wokwi em vez de placa física.** Remove solda e calibração, e o gatilho ao vivo (potenciômetro/botão) é mais confiável que uma lâmpada real.
- **Heurística em vez de RL/Deep Learning.** A política ótima é simples e o *gradient boosting* já está no teto prático de previsibilidade; avaliamos RL e *deep learning* e os

descartamos com justificativa (ver Conclusões).

- **Admissão ciente de risco.** Para *jobs* não interrompíveis, a decisão de iniciar simula o estado de carga ao longo da duração do *job*; essa correção estrutural foi o que tornou a vantagem da previsão robusta.

2.7 Dashboard e demonstração

O *dashboard* (Streamlit) mostra telemetria ao vivo, a curva de previsão, as decisões da IA e o comparativo. A Figura 2 ilustra uma órbita: a potência cai no eclipse, a previsão (tracejada) antecipa a queda, a temperatura fica sob o limite e a bateria drena e recarrega; o MPC faz *checkpoint* antes de a energia acabar. O gatilho ao vivo: baixar o potenciômetro provoca o eclipse, o LED escurece e a decisão aparece no banco com `model_version=hgb`.

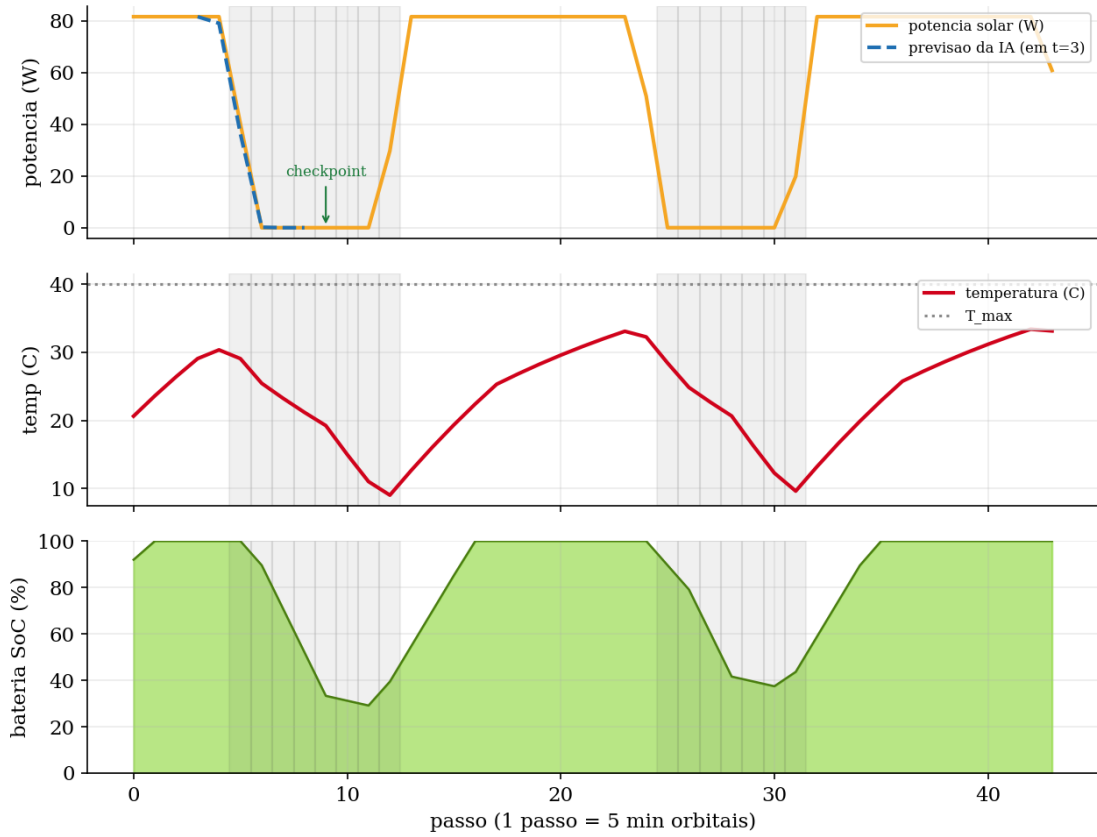


Figura 2: Telemetria e decisão da IA ao longo de duas órbitas (faixas em cinza são eclipses).

A Figura 3 mostra o *dashboard* em execução durante um eclipse: a potência caiu, a previsão (tracejada, em azul) aponta para baixo e a última decisão da IA é *checkpoint*.

3 Resultados Esperados

3.1 O forecaster é aprendizado de verdade

A Tabela 1 mostra o *skill* (1 menos a razão de RMSE) contra dois baselines: persistência inteligente (mesma fase, órbita anterior) e a *efeméride pura* (potência nominal da órbita conhecida). O *skill* positivo contra a efeméride prova que o modelo prevê a parte *incerta* (apontamento),



Figura 3: Dashboard ao vivo (aba “Ao Vivo”), com o nó em eclipse: métricas, telemetria com a curva de previsão e a última decisão (*checkpoint*).

não o eclipse determinístico. O *skill* cai com o horizonte, como esperado, e não é $\sim 1,0$; se fosse, indicaria vazamento.

Alvo / horizonte (passos)	1	2	3	4	5
Energia: skill vs persistência	0,68	0,67	0,66	0,65	0,64
Energia: skill vs efeméride	0,78	0,71	0,67	0,63	0,60
Folga térmica: skill vs persistência	0,89	0,81	0,73	0,67	0,61

Tabela 1: Skill do forecaster na validação por episódio. Sem vazamento (skill longe de 1,0; cai com o horizonte).

3.2 Quando prever ganha de reagir

Para *jobs* não interrompíveis (uma vez iniciado, ou termina ou é perdido se a bateria zerar), comparamos a admissão guiada pela previsão do ML contra a guiada por persistência, usando a *mesma* regra de admissão. A métrica é a pontuação líquida (concluídos menos duas vezes os perdidos). A Figura 4 é a curva de regime: a vantagem da IA é grande e robusta quando a bateria é apertada e desaparece quando há folga.

No regime energia-crítico (bateria de 6 a 12 Wh), a IA conclui cerca de 20 a 25% mais *jobs* e perde de 65 a 75% menos, com diferença de pontuação de +3,3 a +4,9 e intervalo de confiança de 95% acima de zero em três blocos de sementes independentes. Com bateria folgada (≥ 16 Wh), o resultado é um empate, e nós o reportamos. Uma ablação mostrou ainda que uma admissão excessivamente conservadora (quantil P10) zera o *throughput*: aprendemos que o que importava era a correção estrutural (simular o estado de carga ao longo do *job*), não o quantil.

3.3 Impacto esperado

A leitura central é que *a previsão vale proporcionalmente ao custo do erro irreversível*. Em um satélite real, com restrições energéticas apertadas e *jobs* caros de refazer, esse regime é exatamente o esperado, o que sugere que o escalonamento preditivo é mais valioso na prática do

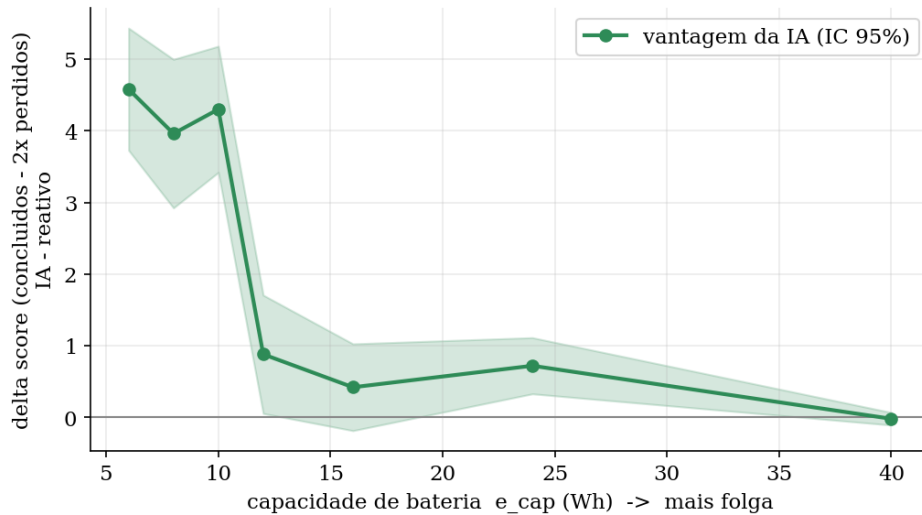


Figura 4: Vantagem da IA sobre o reativo (diferença de pontuação, com IC 95% pareado) em função da capacidade de bateria. A vantagem concentra-se no regime energia-crítico.

que em uma simulação folgada. O sistema é barato, reproduzível e mensurável, o que o torna uma base honesta para evoluir.

4 Conclusões

Construímos e demonstramos, de ponta a ponta, um conceito que hoje só aparece em *papers*: um escalonador de computação orbital ciente de energia e temperatura, com ESP32, sensores, banco de dados e aprendizado de máquina com papel central. Mais do que isso, medimos com rigor científico *onde* a previsão agrega valor (regime energia-crítico) e fomos transparentes sobre o empate no regime folgado.

Por que não RL nem Deep Learning. Avaliamos os dois. A política ótima do escalonador é uma árvore de poucas regras sobre duas variáveis: um agente de *reinforcement learning* no máximo empata, ao custo de instabilidade e do risco de ajustar a recompensa (uma porta para resultados viciados). Para o *forecaster*, com dataset pequeno e horizonte curto, o *gradient boosting* já está no teto de previsibilidade imposto pela incerteza irreduzível; *deep learning* apenas sobreajustaria. Saber o que não usar é parte da engenharia.

Limitações e trabalho futuro. Os dados são simulados (de forma realista: órbita por Kepler, radiação de Stefan-Boltzmann, ruído de sensor). O escalonamento usa uma carga sintética; o próximo passo natural é uma fila combinatória de N *jobs* heterogêneos com prazos, regime em que uma política aprendida (RL) ou de otimização pode finalmente superar a heurística manual. Validar o *forecaster* em hardware real (ESP32 físico) também está no horizonte.

Vídeo de demonstração

Link do vídeo: <https://www.youtube.com/watch?v=GAs3oIsI3PU>

Repositório do projeto:

<https://github.com/lfbardi-don/TEMPLATE-TIA0-2026/tree/main/1TIA0/Global-Solution-1>

Referências

- [1] Starcloud (anteriormente Lumen Orbit). Data centers em órbita, 2025.
- [2] Google. Project Suncatcher: explorando *machine learning* no espaço, 2025.
- [3] Pedregosa et al. Scikit-learn: Machine Learning in Python. JMLR, 2011.